Run Book for ViewCompare

Date

Copyright © 2026
Denodo Technologies Proprietary and Confidential
denodo.com

Contents

Contents 2
Introduction 3
Scope 3
Key Configuration Concepts 4
Prerequisites 4
Setup and Configuration of ViewCompare 5
    Report Table 5
Java Stored Procedure Setup (Checksum Comparison) 6
    Import the JAR 6
    Importing VQL Files: 7
    Setup for Other Databases 8
    Data Source Configuration 8
Executing Your Tests 9
    1. Metadata Comparison 9
    2. Count Check 10
    3. Checksum Comparison 11
History and Persistence Behavior 13
Appendix A: Metadata Comparison 14
        Summary of Observations with Explanations 14
        Sample Output — bv_metadata_comparison 14
Appendix B: Count Check 15
        Summary of Observations with Explanations 15
        Execution Steps 15
        Sample Output — bv_row_count_comparison 15
Appendix C: Checksum Comparison 16
        Observations 16
        Sample Output — bv_checksum_comparison 16

Introduction

Denodo periodically releases both major and minor updates to its platform. During migrations, whether between major versions or incremental updates, customers are required to perform thorough testing to ensure consistency and reliability across environments.

To facilitate and accelerate this process, the ViewCompare has been developed to identify differences in both metadata and data across Denodo environments. The framework enables comparison of view definitions (metadata) as well as the underlying data, helping users detect discrepancies that may arise during migration. It is designed to provide detailed insights, including record-level differences, for a specified set of views across different versions.

In addition to detailed comparisons, the framework offers flexibility by allowing users to perform basic validation checks or opt for more granular, record-level analysis based on their requirements. This adaptability ensures that the framework can cater to varying levels of testing depth.

The ViewCompare can be deployed on any Denodo instance and configured to connect to both source and target environments. This allows users to perform comprehensive validation and View Validation Testing across different systems in a centralized and efficient manner.

Scope

The ViewCompare is designed to assist customers in accelerating the testing process (limited to comparing the views) during migrations to newer Denodo versions or when applying platform updates. It provides a foundational setup that enables efficient comparison of views across Denodo environments, whether within the same server or across different servers.

The framework supports the following types of comparisons between two Denodo environments:

1. Metadata Comparison

Evaluates differences in view definitions, including schema structure, field names, data types, and more.

2. Row Count Comparison

Compares the total number of records returned by each view to identify any discrepancies in data volume.

3. Row-Level Data (Checksum) Comparison

Performs detailed comparison of the actual data returned by the views. Helps identify mismatches at the record level, enabling precise validation of data consistency.

Key Configuration Concepts

The Denodo ViewCompare solution must be configured within a Denodo V9 instance. The overall setup is organized into four key components (or “tiles”), each representing a specific role in the framework:

1. Host VDP server

- The Denodo server where the ViewCompare is deployed and executed.
- This instance orchestrates the comparison process between environments.
- The host must be a Denodo V9 server and it is recommended that the host be a non-production server, such as a Sandbox, to ensure minimal operational impact. The dedicated Host server connects to both the **Source** and **Target** VDP servers.

2. Source VDP server

- Represents the existing or current Denodo environment.
- This serves as the baseline against which the target environment is validated.

3. Target VDP server

- Refers to the new or upgraded Denodo environment.
- This environment is compared against the Source to identify any discrepancies.

4. Report Tables

- A relational database (RDBMS) used to store the results generated by the ViewCompare.
- The stored results remain persisted along with a date_compare field for execution identity.

Prerequisites

The following prerequisites must be fulfilled before setting up the framework:

1. Relational Database

- A relational database must be available to store the output generated by the ViewCompare.
- Ensure that the database is accessible from the environment where the framework will be executed.

2. Custom JAR Files

- **ChecksumCompare.jar** (backs the checksum_compare procedure) and **ChecksumCompareQuery.jar** (backs checksum_compare_by_query), obtained from the custom

jar folder. Both are custom JARs and must be imported and registered as Java stored procedures on the Host VDP server.

3. Java Runtime

- The Host server's JVM must be **Java 17 or later.** Both ChecksumCompare.jar and ChecksumCompareQuery.jar are compiled to Java 17 class-file version 61.

4. Port and Connectivity

- For the ViewCompare to successfully communicate with both the Source and Target VDP servers, it is essential that these servers are reachable from the Host VDP server over the network. Specifically, the Host VDP server must have proper connectivity to the **Source VDP server** and **Target VDP server** and port 9999 (the default VDP port) should be open and accessible on both ends.

5. Primary key for Row-Level Data Comparison

- For the Row-Level Data Comparison to function accurately, it is essential that the views being compared contain a column that uniquely identifies each record. This unique identifier ensures precise matching of rows between the Source and Target datasets during comparison. While this field does not need to be explicitly defined as a primary key within Denodo, it must consistently contain unique, non-null values across the dataset to avoid mismatches or incorrect comparison results.

Setup and Configuration of ViewCompare

The Setup of ViewCompare is done in the Host VDP Server. Follow the steps outlined in each of the sections below.

Report Table

To enable persistence of testing results, the required report tables must be created in a relational database. Please follow the steps below:

1. Select a Supported Database

- Choose any JDBC-supported relational database where the testing results will be stored.
- Ensure that the database is accessible from the Denodo environment (Host VDP server).

2. Execute the Appropriate database DDL Script

- Navigate to the /DDL folder containing DDL scripts for multiple database systems.
- Select the script corresponding to your chosen database (e.g., SQL Server, Oracle, etc.).
- Execute the selected DDL script to create the required report tables in the database of your choice.

3. Database Sizing Consideration

Ensure that sufficient database storage capacity is provisioned for the report tables. ViewCompare **persists the results** of each comparison run, and the report tables may grow over time depending on the number of views being compared and the frequency of test executions. Please consider the expected **data volume and retention requirements** when sizing the database.

Note: We have currently provided DDL scripts for SQL Server, MySQL, and Oracle. If you require creating the report tables in a different database, we kindly request you to use these DDLs as a reference and make the necessary modifications to suit your target system.

Java Stored Procedure Setup (Checksum Comparison)

Checksum Comparison runs entirely inside the Denodo VDP engine as a Java stored procedure.

Import the JAR

- Download the JAR corresponding to the input style you plan to use: ChecksumCompare.jar (compare by view name) and/or ChecksumCompareQuery.jar (compare by SELECT query).
- In Design Studio, import the JAR file (File > Extension Management) and create a new Java stored procedure (File > New) using the class name shown below.

JAR	Class Name	Registered Procedure
ChecksumCompare.jar	com.denodo.vdp.custom.functions.ChecksumCompare	checksum_compare
ChecksumCompareQuery.jar	com.denodo.vdp.custom.functions.ChecksumCompareQuery	checksum_compare_by_query

Grant Required Permissions

- The executing user needs read access to the views (or the views/tables referenced by the SELECT queries) being compared.
- The executing user needs insert rights on bv_checksum_comparison.

Confirm the Report View

After importing the below-mentioned VQL files, you will find a database vdb_viewcompare which contains writable base views. The view bv_checksum_comparison with columns source_view_name, target_view_name, primary_key, observation, date_compare, source_query and target_query mapped through a JDBC data source to the report table

Note: When upgrading from a previous version, the required custom JAR files must be imported into the Host VDP server and the corresponding Java stored procedures must be created.

Importing VQL Files:

Within the provided deliverable, you will find two distinct VQL files required for the setup. Please follow the steps below to complete the import process:

1. Download and Extract Files

Begin by downloading the vql_export.zip file. Once downloaded, extract its contents to the folder in the Host server. This has the view logic and code required for this implementation.

Similarly, you will find a vql_report_views_<JDBC_database>.zip which contains the VQL of the views needed for storing the results. Please download and import the appropriate VQL based on your preferred JDBC datasource.

2. Update the Properties File

Locate the vql_export > elementExport.properties file and update it with the appropriate configuration values:

- Modify the connection parameters so that they correctly point to the data source where the report tables have been created.
- Encrypt the source connectivity password using the appropriate script:
    - On Windows: DENODO_HOME/bin/encrypt.bat
    - On Linux: DENODO_HOME/bin/encrypt.sh
- After encryption, replace the parameter in the properties file with the encrypted value.

Similarly, updated the vql_report_views_<JDBC_database>.properties file to reflect the corresponding JDBC connection details.

3. Import the VQL:

- Import the vql_export.vql file along with the updated properties file to ensure the configuration is applied correctly. During the import process, use the password admin when prompted.
- Finally, download and import the vql_report_views_<JDBC_database>.zip file along with the updated properties file.

Note: The Python-related views and their dependencies from the previous version are no longer required for the current ViewCompare implementation and can be left unused.

If you are upgrading from a previous version of ViewCompare, perform a **source refresh for the Report Views** to ensure they point to the appropriate report tables and schema.

Setup for Other Databases

Note: If the report tables were created in a database other than MSSQL, Oracle, or MySQL, please follow the steps below to ensure proper integration with the framework.

Steps to Follow

1. Create a Datasource Connection

- Set up a new datasource connection in the vdb_viewcompare VDB.
- This connection should point to the database where the report tables were created.

2. Create Base Views

- Once the connection is established, create base views on top of the report tables created in the previous step
- Ensure that the base views strictly follow the required naming standards:

Table Name	Base View Name
metadata_comparison	bv_metadata_comparison
row_count_comparison	bv_row_count_comparison
checksum_comparison	bv_checksum_comparison

Important: Following the above naming conventions is mandatory. This ensures that the framework can correctly identify and utilize the views without any issues.

Data Source Configuration

Please configure the following data sources to ensure proper connectivity with the respective environments:

1. Source Denodo Server Connection

- **ds_source_denodo_server**: Configure this data source to establish a connection with the **Source VDP server**
- Ensure that all connection parameters (such as host, port, database name, and credentials) accurately reflect the **Source VDP server**.
- Provide appropriate values for the aforementioned parameters, as outlined below:
    - **Host** – Specify the hostname of the Source VDP server.
    - **Port** – Specify the port number on which the Source VDP server is running.
    - **Database Name** – Specify the name of the Virtual Database used to establish the connection.

      The above parameters must be configured within the JDBC URI.

    - **Credentials** - Provide valid username and password details in the user configuration to authenticate and access the Source VDP server.

2. Target Denodo Server Connection

- **ds_target_denodo_server**: Configure this data source to connect to the **Target VDP server**.
- Ensure that all connection parameters (such as host, port, database name, and credentials) accurately reflect the **Target VDP server**.
- Provide appropriate values for the aforementioned parameters, as outlined below:
    - **Host** – Specify the hostname of the Target VDP server.
    - **Port** – Specify the port number on which the Target VDP server is running.
    - **Database** Name – Specify the name of the Virtual Database used to establish the connection. The above parameters must be configured within the JDBC URI.
- **Credentials** - Provide valid username and password details in the user configuration to authenticate and access the Target VDP server.

Executing Your Tests

1. Metadata Comparison

Execute the PL/VQL procedure metadata_check from the Host VDP Server with the required input parameters. Results are available in the view bv_metadata_comparison. See Appendix A for a description of the observation categories and a sample output.

Syntax — Input as View Name

```
CALL metadata_check(
   'source_dbname','source_view_name','target_dbname',
   'target_view_name');
```

Note: all four inputs are VARCHAR and mandatory. Output: summary.

Example:

```
CALL metadata_check(
   'admin','bv_employees','admin',
   'bv_employees');
```

2. Count Check

Execute count_check (by view name) or count_check_by_query (by SELECT statement) from the Host VDP Server. Results are available in the view bv_row_count_comparison. See Appendix B for a description of the result types and a sample output.

A. Syntax — Input as View Name

```
CALL count_check(
  'source_db_name','source_view_name','target_db_name',
  'target_view_name');
```

Note: all four inputs are VARCHAR and mandatory. Output: summary.

B. Syntax — Input as Query

```
CALL count_check_by_query('source_query', 'target_query');
```

Note: Both inputs are VARCHAR and mandatory. Output: summary.

Example:

```
CALL count_check_by_query(
  'SELECT * FROM admin.bv_test_products',
  'SELECT * FROM admin.bv_test_products2');
```

C. Syntax — Input as Query with Condition

```
CALL count_check_by_query('source_query', 'target_query');
```

Example:

```
CALL count_check_by_query(
  'SELECT * FROM admin.bv_test_products WHERE "Product"=''Z8Z04A''',
  'SELECT * FROM admin.bv_test_products2 WHERE "Product"=''Z8Z04A''');
```

3. Checksum Comparison

Checksum Comparison detects row-level differences between a source and a target dataset and persists each difference to bv_checksum_comparison. Two stored procedures are available, covering three input styles.

Procedure	Input Style	Parameters	Result View
checksum_compare	Two views, referenced by name	pk_field_src, view_name_src, db_name_src, pk_field_tgt, view_name_tgt, db_name_tgt — all VARCHAR, mandatory	bv_checksum_comparison
checksum_compare_by_query	Two SELECT queries	pk_field_src, source_query, pk_field_tgt, target_query — all VARCHAR, mandatory	bv_checksum_comparison
checksum_compare_by_query	Two SELECT queries, each with a WHERE condition	Same as above; query text may include a WHERE clause	bv_checksum_comparison

How Each Procedure Compares Data

	checksum_compare	checksum_compare_by_query
Input	Two views: database name, view name, and PK column for each side	Two SELECT queries and the PK column name for each
How it compares	Validates that each view and PK column exists (via GET_VIEW_COLUMNS()), then reads pk, hash_inp from vdb_viewcompare.source_checksum() and target_checksum()	Runs both queries and computes a SHA-256 hash per row over all non-PK columns (NULL-safe, trailing spaces trimmed on strings)

Both procedures classify each row by matching primary keys, and both write one row per difference to bv_checksum_comparison:

- missing in target — the PK exists in the source but not in the target.
- missing in source — the PK exists in the target but not in the source.

- mismatch — the PK exists on both sides, but the row's non-PK content differs.

Identical rows are not written. See Appendix C for a sample output.

A. Syntax — Input as View Name

```
CALL checksum_compare(
  'pk_field_src','view_name_src','db_name_src',
  'pk_field_tgt','view_name_tgt','db_name_tgt');
```

Note: all six inputs are VARCHAR and mandatory. Output: summary.

B. Syntax — Input as Query

```
CALL checksum_compare_by_query(
  'pk_field_src','SELECT ... FROM db.view_a',
  'pk_field_tgt','SELECT ... FROM db.view_b');
```

Note: all four inputs are VARCHAR and mandatory. Output: summary.

C. Syntax — Input as Query with Condition

```
CALL checksum_compare_by_query(
  '<pk_column_src>',
  '<SELECT col_list FROM <db>.<view> WHERE <condition>>',
  '<pk_column_tgt>',
  '<SELECT col_list FROM <db>.<view> WHERE <condition>>'
);
```

Example:

```
CALL checksum_compare_by_query(
  'CASE_ID',
  'SELECT * FROM denodo.bv_cc_case_definition WHERE case_comments=''No comments''',
  'Case_Id',
  'SELECT * FROM denodo.bv_cc_case_definition WHERE case_comments=''No comments'''
);
```

D. **Table Aliases and Primary Key Matching (checksum_compare_by_query)**

When a query uses table aliases (for example SELECT a.* FROM db.view_a a JOIN ...), the result columns may be returned with the alias in front of the name, such as a.serialnumber. The procedure handles this as follows:

- Only the column name is used to find the primary key. An alias in the primary key parameter or in the result column name is ignored. serialnumber, a.serialnumber and "a"."serialnumber" all refer to the same column. Matching is not case-sensitive.
- The alias does not select a table. Passing a.serialnumber does not make the procedure prefer table a over table b. If a query returns several columns with the same name, all of them are treated as primary key columns.
- Duplicate primary key columns must hold the same value in every row. For example, an inner join on a.serialnumber = b.serialnumber returns two equal columns, which is accepted. If the values differ in any row, the procedure stops with an “ambiguous” error. In that case, select explicit columns in the query.
- All primary key columns are excluded from the checksum. Only the remaining columns are compared.
- Both queries must return the same columns in the same order. Every non-key column is compared in column order. If one query returns extra columns (for example SELECT * over a join, compared with a single view), every row is reported as a mismatch even when the data is identical. Use SELECT a.* or list the columns explicitly so that both sides match.
- Column names that contain a dot are not supported. The text after the last dot is taken as the column name.
- If the primary key column is not found, the error message lists the columns that the query returned.

Example:

```
CALL checksum_compare_by_query(

'serialnumber',

'SELECT a.* FROM admin.view_a a INNER JOIN admin.view_b b ON a.serialnumber =
b.serialnumber',

'serialnumber',

'SELECT * FROM admin.view_a'

);
```

E. Query text size and LABEL_MAX_LENGTH

For checksum_compare_by_query, source and target query text are stored in the report table. The amount of query text that can be retained depends on the database column type, byte/character semantics, character set, and database limits. Configure LABEL_MAX_LENGTH to match the selected column capacity. The figures below are examples and should be validated against the database version and the DDL used in your deployment.

Database / column type	Approximate query text capacity	LABEL_MAX_LENGTH
MySQL – VARCHAR(2000)	About 1,997 characters + ellipsis for plain English	2000
MySQL – VARCHAR(16000)	About 15,997 characters + ellipsis; subject to row/column limits	16000
MySQL – TEXT	Up to 65,535 bytes; multibyte text may fit fewer characters	65535
Oracle – VARCHAR2(4000), STANDARD	Up to 4,000 bytes; about 3,997 characters + ellipsis for plain English	4000
Oracle – CLOB	Large-text capacity; choose and test a suitable implementation limit	Set per implementation
SQL Server – VARCHAR(8000)	Up to 8,000 bytes; about 7,997 characters + ellipsis for plain English	8000
SQL Server – VARCHAR(MAX)	Up to approximately 2 GB	Set per implementation

Disclaimer: Query text may be truncated if it exceeds the configured limit. Choose a column type and LABEL_MAX_LENGTH that accommodate the largest expected query, then test long queries end to end. For very large query text, use a supported large-text type and confirm compatibility with the procedure and JDBC driver.

Database references

MySQL: https://dev.mysql.com/doc/refman/8.0/en/storage-requirements.html

Oracle: https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/datatype-limits.html

SQL Server: https://learn.microsoft.com/en-us/sql/t-sql/data-types/char-and-varchar-transact-sql

History and Persistence Behavior

Every comparison procedure — metadata_check, count_check / count_check_by_query, and checksum_compare / checksum_compare_by_query simply inserts its results into the corresponding report view (bv_metadata_comparison, bv_row_count_comparison, bv_checksum_comparison).

None of them clear the table first, so results from every prior run remain in place alongside new rows, indefinitely, across all three comparison types.

Always scope result queries by date_compare and, where useful, by the relevant view/query identifiers, rather than reading a result view unfiltered.

Example:

```
SELECT *
 FROM vdb_viewcompare.<result_view>
 WHERE date_compare >= '<run start timestamp>'
 ORDER BY date_compare DESC;
```

If a clean table is required between test cycles, run an explicit TRUNCATE TABLE against the underlying report table before the run — none of the stored procedures do this automatically.

Appendix A: Metadata Comparison

Summary of Observations with Explanations

Category	Details
datatype or precision	Mismatches in column_vdp_type, column_jdbc_type, column_size, etc.
type decimal, column radix, primary key definition, null value definition, field description, autoincrement definition	All these values are self-explanatory; refer to the GET_VIEW_COLUMNS documentation for additional detail.

Sample Output — bv_metadata_comparison

source_view_name	target_view_name	column_name	source_datatype	target_datatype	observation	date_compare
bv_employees	bv_employees	department	NVARCHAR	NVARCHAR	mismatch in "field description"	2026-09-22 07:45:12
bv_employees	bv_employees	firstname	NVARCHAR	NVARCHAR	mismatch in "datatype or precision" "null value definition"	2026-09-22 07:45:12
bv_employees	bv_employees	lastname	NVARCHAR	TIMESTAMP	mismatch in "datatype or precision" "null value definition"	2026-09-22 07:45:12

Appendix B: Count Check

Summary of Observations with Explanations

Result Type	Description
Matching	Row counts are equal in both views.
Mismatch	Row counts differ between source and target.

Execution Steps

- Execute the PL/VQL procedure count_check (or count_check_by_query) from the Host VDP Server with the required input parameters.
- Results will be available in the view bv_row_count_comparison.

Sample Output — bv_row_count_comparison

Input as View Name

source_view_name	target_view_name	source_row_count	target_row_count	observation	date_compare
bv_employees	bv_employees	6	6	matching	2026-09-22 07:45:12

Input as Query

source_view_name	target_view_name	source_query	target_query	source_row_count	target_row_count	observation	date_compare
null	null	SELECT * FROM admin.test_products WHERE "product"='Z8Z04A'	SELECT * FROM admin.test_products2 WHERE "product"='Z8Z04A'	6	6	matching	2026-09-22 07:45:12

Appendix C: Checksum Comparison

Observations

Observation Type	Description

missing in source	Records present in the target but not in the source.
missing in target	Records present in the source but not in the target.
mismatch	Records exist on both sides, but the data differs.

Sample Output — bv_checksum_comparison

A. Input as View Name

source_view_name	target_view_name	source_query	target_query	primary_key	observation	date_compare
bv_employees	bv_employees	null	null	7	missing in source	2026-09-22 07:45:12
bv_employees	bv_employees	null	null	6	missing in target	2026-09-22 07:45:12
bv_employees	bv_employees	null	null	3	mismatch	2026-09-22 07:45:12

B.Input as Query

source_view_name	target_view_name	source_query	target_query	primary_key	observation	date_compare
null	null	Select * from viewname	Select * from viewname	7	missing in source	2026-09-22 07:45:12
null	null	Select * from viewname	Select * from viewname	6	missing in target	2026-09-22 07:45:12
null	null	Select * from viewname	Select * from viewname	3	mismatch	2026-09-22 07:45:12

Appendix D: Upgrading from the Python-Based Version

This appendix is for customers who already run the earlier, Python-based ViewCompare (checksum comparison executed through an SSH Custom Wrapper and **checksum_py_script.py**) and are moving to the Java-based implementation described in this Run Book. Complete the steps in order. Do not remove any Python component until the Java-based flow has been validated (step 7).

What Changes

Area	Python-based version	Java-based version
Checksum execution	SELECT * FROM bv_ssh_python_script_executor WHERE ...	CALL checksum_compare(...) or CALL checksum_compare_by_query(...)
Runs in	External Python script over SSH	Java stored procedures inside the Host VDP engine
Host requirements	Python 3.10+, JayDeBeApi, SSH server (port 22), SSH Custom Wrapper	Java 17 or later on the Host JVM; ChecksumCompare.jar and ChecksumCompareQuery.jar
Result retention	Report tables truncated on each run; only the latest run is kept	Results are never cleared; every run is retained and identified by date_compare
Input styles	One view per side	Views, or SELECT queries (with optional WHERE condition)
Report columns	Earlier column set	Adds date_compare and, for row count and checksum, source_query and target_query

Upgrade Steps

1. **Check the Java version.** The Host VDP server JVM must be Java 17 or later (the JARs are compiled to class-file version 61).
2. **Update the report tables.** Compare your existing metadata_comparison, row_count_comparison and checksum_comparison tables with the DDL script for your database in the /DDL folder. Add the missing columns (date_compare, and source_query / target_query where applicable), or recreate the tables from the new DDL. Export any results you want to keep first. Size the database for accumulating results (see Report Table).
3. **Import the JARs and register the procedures.** Import ChecksumCompare.jar and ChecksumCompareQuery.jar (File > Extension Management), then create the Java stored procedures checksum_compare and checksum_compare_by_query using the class names listed under Java Stored Procedure Setup.
4. **Re-import the VQL.** Import the new vql_export and vql_report_views_<JDBC_database> packages with their updated properties files, as described under Importing VQL Files.
5. **Refresh the report views.** Perform a source refresh on bv_metadata_comparison, bv_row_count_comparison and bv_checksum_comparison so they match the updated report tables.
6. **Grant permissions.** The executing user needs read access to the views being compared and insert rights on bv_checksum_comparison.

7. **Validate.** Run metadata_check, count_check and checksum_compare on at least one view that was previously tested with the Python version and confirm the results agree with the earlier run.
8. **Update scripts and procedures.** Replace any job, script or document that calls bv_ssh_python_script_executor with the CALL syntax. Change any query on a result view to filter on date_compare (see History and Persistence Behavior).

Cleanup of Python-Based Components

The Python components are no longer used. After step 7 succeeds, remove them to avoid confusion and reduce the host footprint. Before removing a shared component (Python, SSH server, SSH Custom Wrapper), confirm that no other application or Denodo use case depends on it.

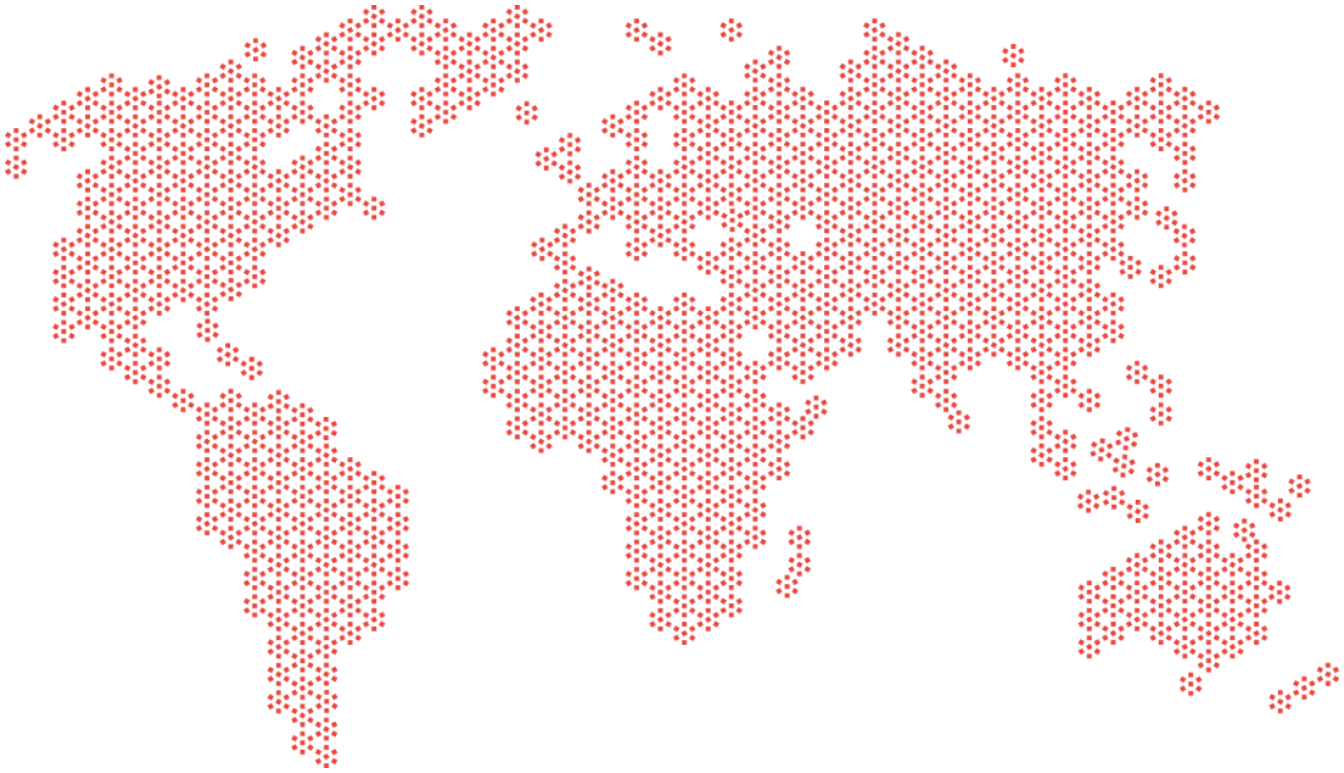
Component	Action	Notes
bv_ssh_python_script_executor (base view)	Drop	Located in the ViewCompare VDB; no longer called by any procedure.
ds_ssh_script_executor (data source)	Drop	Drop the base view first, as it depends on this data source.
SSH Custom Wrapper (Denodo Connect)	Remove if unused elsewhere	Only needed by the old SSH-based execution.
checksum_py_script.py	Delete from the host	Also delete any copy kept in other directories or schedulers.
denodo-vdp-jdbcdriver.jar copied next to the script	Delete the copy	Do not touch the driver under DENODO_HOME.
Python 3.10+ and JayDeBeApi	Uninstall if unused elsewhere	Applies only if installed for ViewCompare.
SSH server on the Host VDP server	Disable if unused elsewhere	Close port 22 if it was opened only for this purpose.
Scheduled jobs or scripts calling the Python flow	Update or remove	Replace with the CALL syntax from step 8.

Cleaning Old Results

The Java-based procedures do not truncate the report tables. If results from the Python-based runs are no longer needed, archive them and run an explicit TRUNCATE TABLE on the underlying report tables before the first Java-based run:

```
TRUNCATE TABLE <metadata_comparison>;
TRUNCATE TABLE <row_count_comparison>;
TRUNCATE TABLE <checksum_comparison>;
```

Otherwise, rows from earlier runs remain and should be excluded by filtering on date_compare.



NOTE: *This document is confidential and proprietary of Denodo. No part of this document may be reproduced in any form by any means without prior written authorization of Denodo.*

denodo

Visit **www.denodo.com** Email **info@denodo.com** Discover **community.denodo.com**

Run Book for ViewCompare